Features left out or removed — and why

Timing side-channel testing [REMOVED]

What it was: — a mode that measured, in nanoseconds, how long the app took to compare a secret, to detect “non-constant-time” code that leaks the secret through timing.

Why it was removed: — on real consumer hardware the tiny timing signal (nanoseconds) was drowned out by ordinary phone background noise (scheduling, heat, other apps), so it only detected the planted flaw some of the time. It also required rebuilding the target app with special test code added — which breaks the whole “works on any app you don't control” promise. It was replaced by the auth-state / response oracle above, which tests a real side channel that is deterministic, needs no app rebuild, and fires every run.

Physical side channels (power / electromagnetic / acoustic / cache)

Out of scope. — These require lab equipment or root-level access to hardware. They are outside what a pure software, no-root tool can honestly measure.

Machine learning in the statistics module

Deliberately excluded. — At realistic test-run counts there isn't enough data to train a meaningful model, so the tool uses transparent, robust statistics instead of ML — honest about what the numbers can support.

The Features

Manifest analysis

Reads AndroidManifest.xml — Flags a debuggable build (ships with a developer backdoor), allowBackup enabled (private data can be copied off the device), and exported components with no permission guard (app screens/services left open to other apps).

Biometric-implementation scan

DEX string-pool identifier scan — Looks inside the app's compiled code for tell-tale names. The key one: “boolean-only authentication” — the app trusts a simple “fingerprint OK” yes/no signal without binding it to a cryptographic key, which attackers can bypass. It reads the code's name table directly rather than fully decompiling, so this is fast even on large apps.

IPC / exported-component authorization oracle (headline check)

Probes exported components for auth bypass. — Asks the sharpest question: can a feature that is supposed to sit behind the fingerprint prompt be opened directly, skipping login entirely? If the “secret” screen launches without authenticating, the biometric gate is decorative.

Auth-state / response oracle (side-channel test)

Distinguishable-response side channel — A more subtle attack. It sends an exposed component a valid-looking identifier and an obviously-invalid one and checks whether the app answers them

differently. If it does, that difference is a leak an attacker can exploit to guess valid usernames or brute-force a token — a classic “oracle” side channel. Unlike timing attacks, this is deterministic: a different answer is a hard fact, so it fires reliably every run.

Logcat leakage observer

Scans system logs for secrets — Reads the phone's log stream for tokens, passwords, keys, or “authentication succeeded” messages the app should never have written down.

allowBackup extraction check

Backup-extractability check — If backups are allowed, private app data can be pulled off the device with a standard tool and no root — flagged so the developer turns backups off for sensitive data.

Severity & recommendation engine

Ranks findings Critical→Info and attaches a concrete fix to each. Lower-confidence static findings are automatically dialled back one step.

AI explanation layer (Gemini)

Turns each confirmed finding into a readable explanation and remediation. It is grounded (given a curated knowledge base so it cites real guidance, not guesses) and only ever explains — it never decides what is a vulnerability. Strips tokens/keys out of the evidence before anything is sent to the AI, so real secrets never leave the machine.

HTML / PDF reports

Each assessment exports a shareable report of all findings.